

Einrichten der Praktikums Umgebung

Dieser Guide erklrt die Einrichtung der Praktikums Umgebung.

Inhaltsverzeichnis

Hardware und Software	2
Anlegen eines STM32 Projekts	3
Pinbelegung	4
Anpassen der Pinbelegung im CubeMX-Tool	5
Konsole fr die Ausgabe	8
Assembler-Programmierung	9
C-Programmierung	12
Funktion: printf	12

Hardware und Software

Für den Praktikumsaufbau werden folgende Hardware- und Softwarekomponenten benötigt.

Hardware

- STM32L476RG Nucleo Board
- 8 rote LEDs
- 8 grüne LEDs
- 8 Kippschalter
- 1 Analog-Potentiometer
- 1 Digital-Potentiometer (Drehgeber)

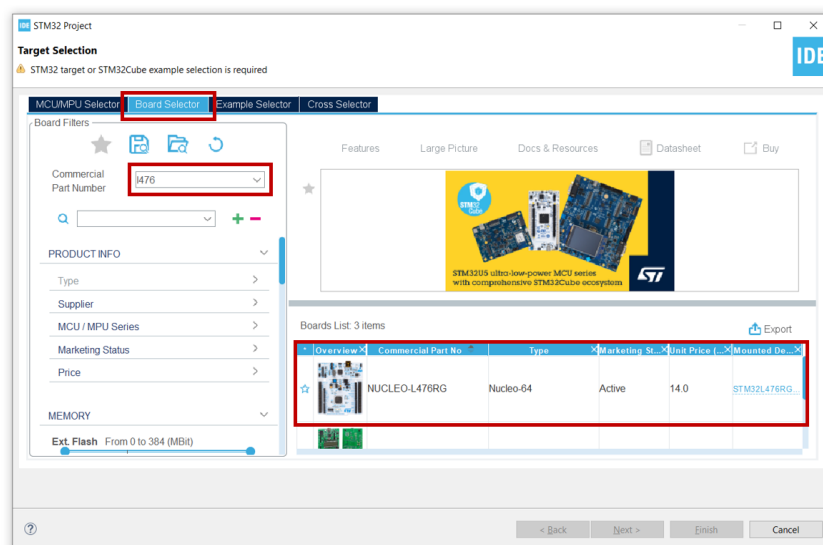
Software

- STM32CubeIDE (verfügbar für Windows, Linux und macOS unter <https://www.st.com/en/development-tools/stm32cubeide.html>)

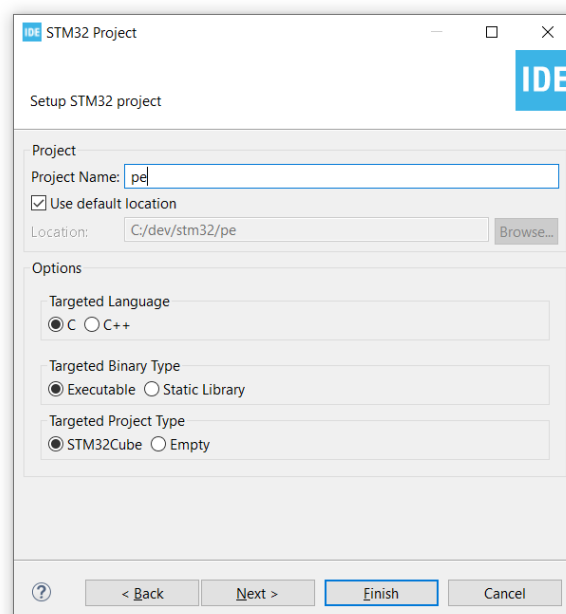
Anlegen eines STM32CubeIDE Projekts

Um ein neues STM32CubeIDE Projekt anzulegen, starten Sie zunächst die IDE.

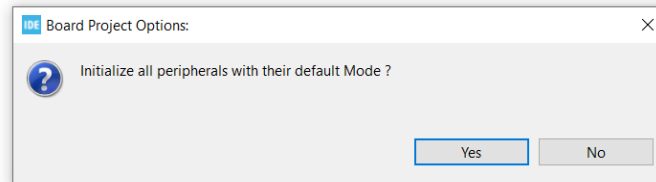
Wählen Sie *File* → *New* → *STM32 Project*. Unter *Target Selection* → *Board Selector* wählen Sie das Board *Nucleo-L476RG* aus. Über die Suchfunktion kann das entsprechende Board schnell gefunden werden.



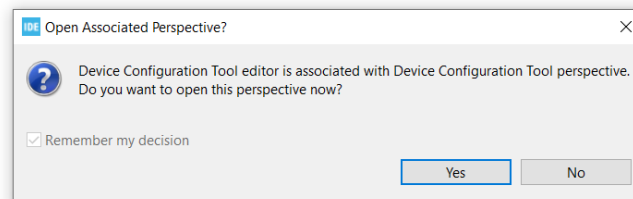
Wählen Sie einen Projektnamen und bestätigen Sie mit *Finish*.



Die IDE fragt, ob die Peripherie mit den Default-Einstellungen initialisiert werden soll. Bestätigen Sie dies.



Die IDE fragt, ob in die CubeMX-Perspektive gewechselt werden soll. Bestätigen Sie auch dies.



Ein neues STM32 Projekt wurde angelegt und es kann mit den Einstellungen für die Pin-Belegung fortgefahren werden.

Pinbelegung

Die Pinbelegung an den Praktikumsaufbauten ist wie folgt. Die LEDs sind an GPIOC, Pin 0 bis 15 (PC0 bis PC15) angeschlossen, die Schalter an GPIOB, Pin 8 bis 15 (PB 8 bis PB15). Das analoge Potentiometer ist an GPIOB, Pin 0 (PB0) angeschlossen. Das digitale Potentiometer (Drehgeber) ist an GPIOA, Pin 0 und 1 angeschlossen (PA0 und PA1). GPIOA, Pin 2 und 3 (PA2 und PA3) dienen der UART Kommunikation.

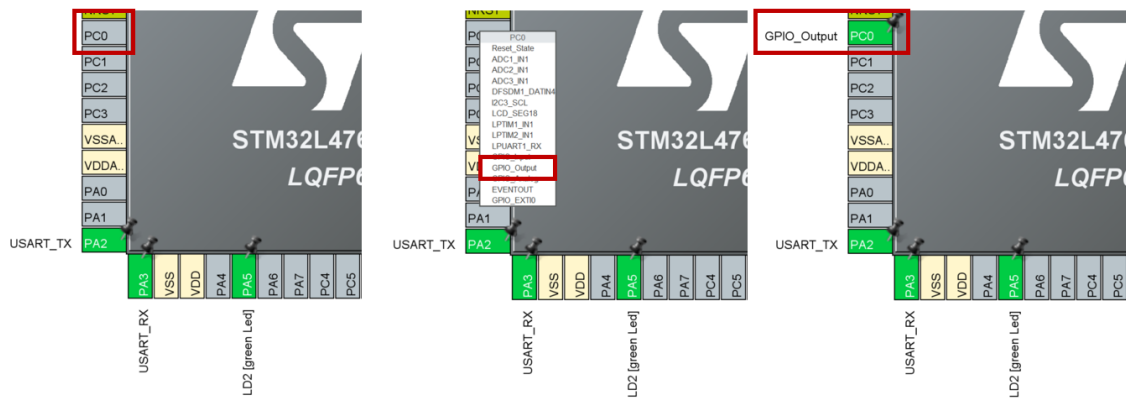
GPIOA	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
													UART		D	

GPIOB	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
																A

GPIOC	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0

Anpassen der Pinbelegung im CubeMX-Tool

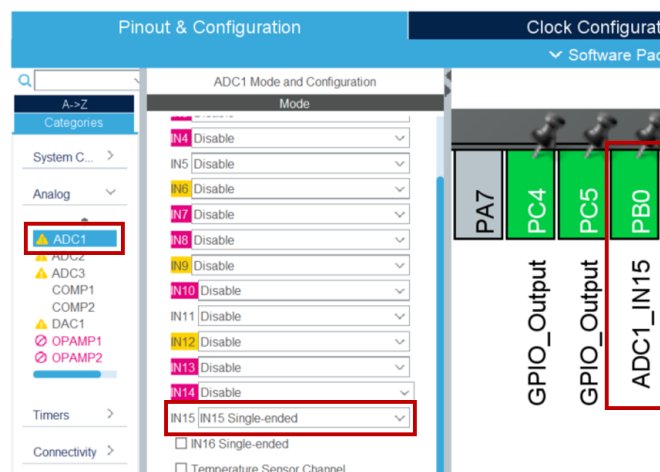
Öffnen Sie die CubeMX Perspektive durch einen Doppelklick auf die ioc-Datei im Project Explorer auf der linken Seite. Durch einen Klick auf den jeweiligen Pin werden die Belegungsoptionen angezeigt.



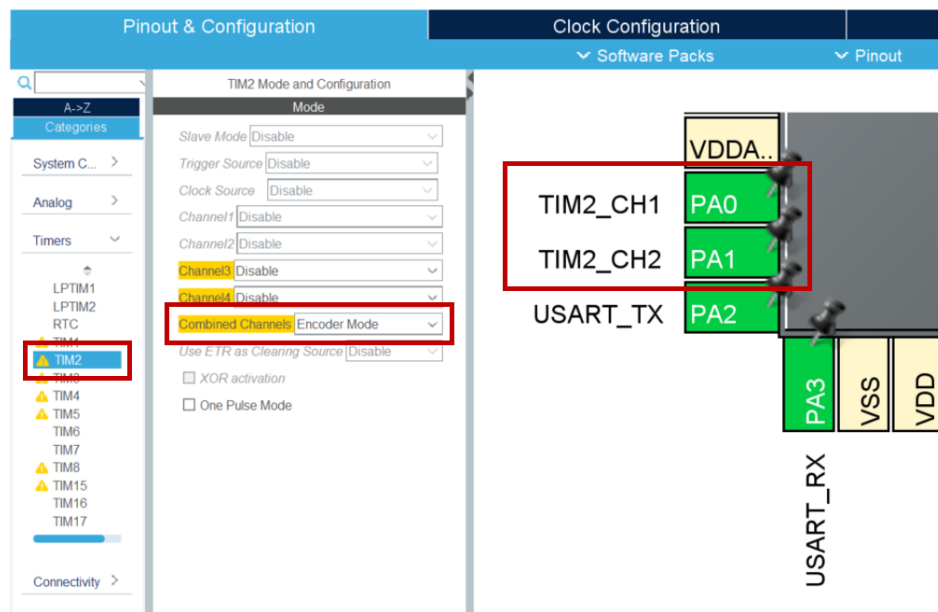
Die Pins **PC15 bis PC0** müssen als **GPIO_Output** konfiguriert werden. Dazu wird auf einen entsprechenden Pin geklickt und die Option **GPIO_Output** wird ausgewählt. Durch ein erneutes Klicken auf die ausgewählte Option kann die Auswahl auch wieder rückgängig gemacht werden.

Die Pins **PB15 bis PB8** müssen als **GPIO_Input** konfiguriert werden.

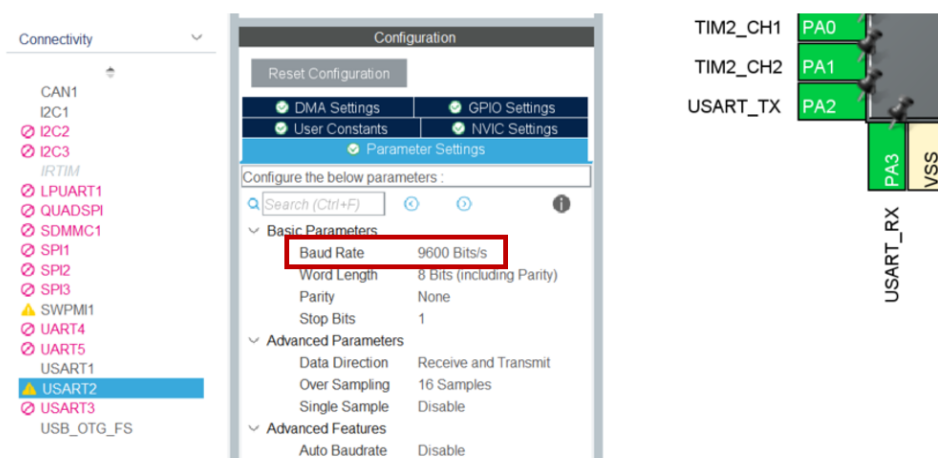
Pin **PB0** muss als **ADC1_IN15** konfiguriert werden. Auf der linken Seite unter *Analog* → *ADC1* muss dabei für den Eingang *IN15* die Einstellung **IN15 Single-ended** ausgewählt werden.



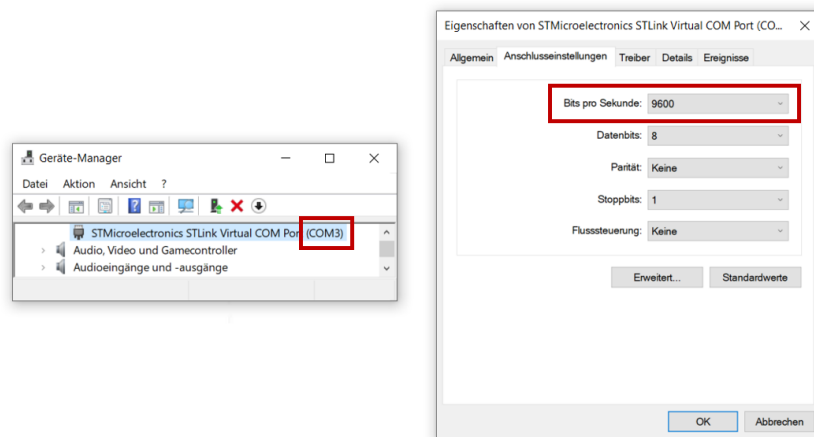
Pin **PA0** muss als **TIM2_CH1** und **PA1** als **TIM2_CH2** konfiguriert werden. Unter *Timers* → *TIM2* muss dafür bei *Combined Channels* der **Encoder Mode** ausgewählt werden.



Die Pins **PA2** und **PA3** werden automatisch für die UART-Kommunikation als **USART_TX** und **USART_RX** konfiguriert, wenn die Default-Initialisierung der Peripherie bei der Erstellung des Projekts bestätigt wurde. Auf der linken Seite unter *Connectivity* → *USART2* können weitere Einstellungen wie z.B. für die Baudrate vorgenommen werden.



Diese Einstellungen müssen mit den Einstellungen des Mikrocontrollers übereinstimmen (ggf. im Gerätemanager überprüfen).



Wenn das Projekt gespeichert wird, wird automatisch der entsprechende C-Code erzeugt. Die Einstellung der Pinbelegung ist somit abgeschlossen.

Die folgende Abbildung zeigt abschließend die gesamte fertige Pinbelegung in der CubeMX-Ansicht.

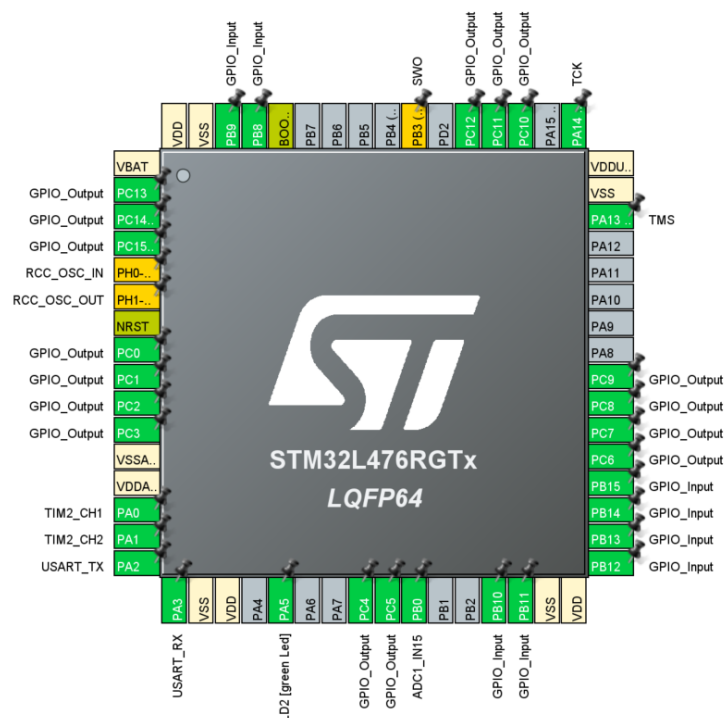


Abbildung 1: Pinbelegung im CubeMX-Tool

Konsole für die Ausgabe

Wenn ein externes Terminal geöffnet wird, müssen auch hier die gleichen Einstellungen für die UART-Kommunikation vorgenommen werden wie in der ioc-Datei.

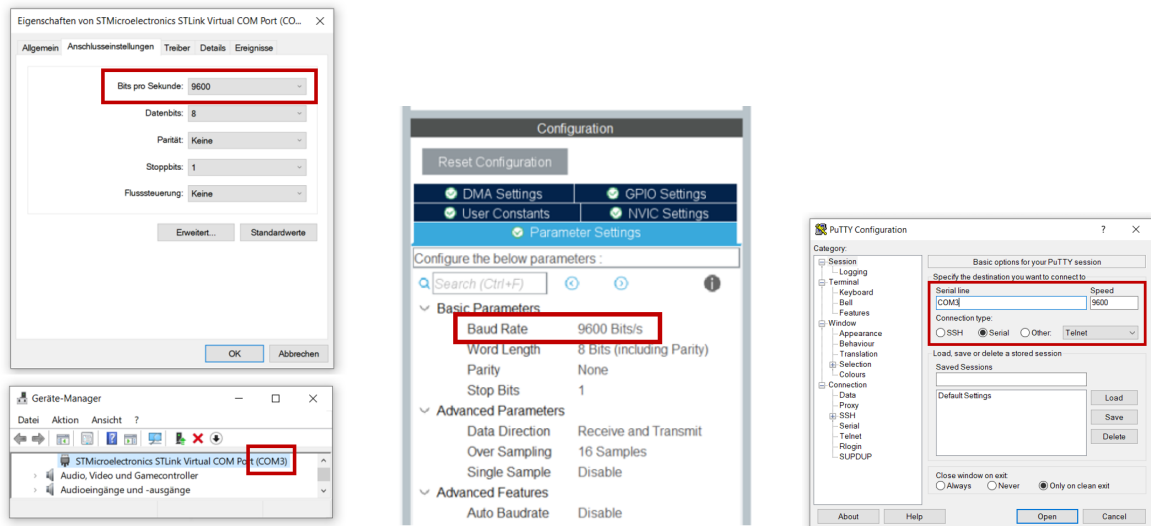


Abbildung 2: Einstellung des Ports und der Baudrate im Gerätemanager, der STM32CubeIDE und der externen Konsole

Außerdem kann eine Konsole in der IDE geöffnet werden, wobei auch hier auf die richtigen Einstellungen zu achten ist. Eine Konsole öffnet man hier über *Open Console* → *Command Shell Console*.

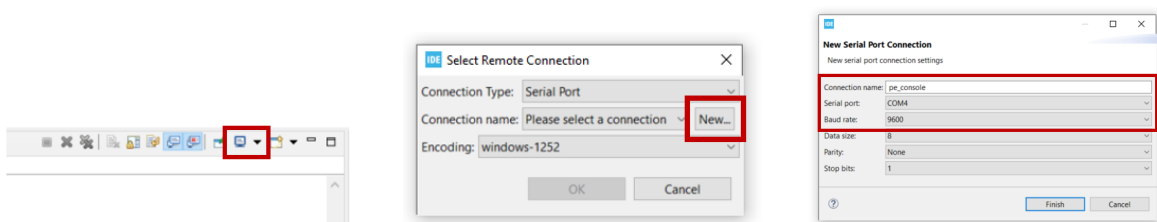


Abbildung 3: Öffnen der Konsole in der STM32CubeIDE

Assembler-Programmierung

Um das Nucleo-Board in Assembler zu programmieren, muss in der `main.c`-Datei zwischen den PFP-Tags je eine externe Funktion für eine Assembler-Funktion definiert werden.

```
/* USER CODE BEGIN PFP */  
extern void beispiel();  
/* USER CODE END PFP */
```

Im Project Explorer auf der linken Seite wird unter *Core* → *Src* über *New* → *File* eine neue Datei angelegt. In diesem Beispiel wird sie mit *beispiel.s* benannt.

Inhalt der Datei *beispiel.s*:

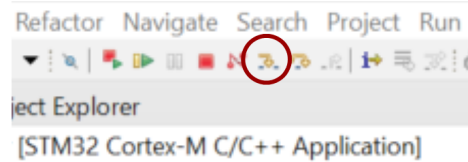
```
.syntax unified  
.global beispiel  
.text  
beispiel:    ldr r0, =0x20000000  
             ldr r1, =0x11111111  
             ldr r2, =0x22222222  
             mov r1, r2  
             str r2, [r0]  
  
             @ Ende des Programms  
             @ Stop durch Endlos-Schleife  
ende:       nop  
            b     ende
```

Danach kann die Assembler-Funktion aufgerufen werden.

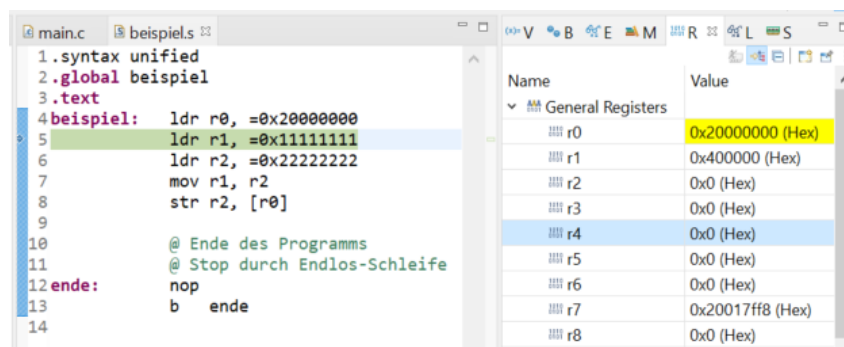
```
/* USER CODE BEGIN 2 */  
beispiel();  
/* USER CODE END 2 */
```

Das Debugging kann über einen Rechtsklick auf den Projektnamen und *Debug As* → *1 STM32 Cortex-M C/C++ Application* gestartet werden. Wenn ein Breakpoint vor dem Einstieg in die Assembler-Funktion gesetzt wird, kann diese Schritt für Schritt ausgeführt werden.

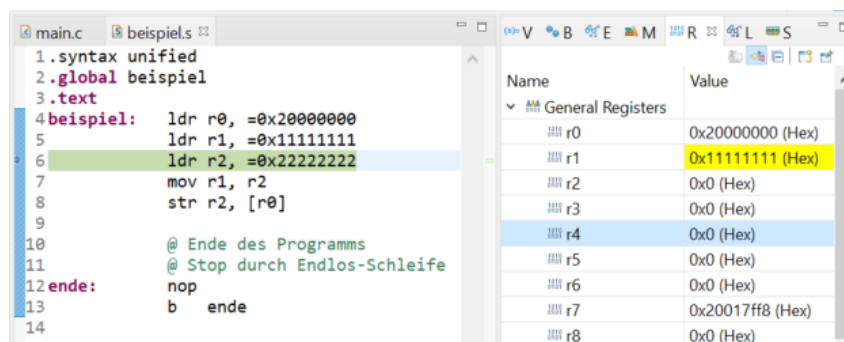
```
112 /* Initialize all configured peripherals */
113 MX_GPIO_Init();
114 MX_USART2_UART_Init();
115 /* USER CODE BEGIN 2 */
116 beispiel();
117 /* USER CODE END 2 */
118
119 /* Infinite loop */
120 /* USER CODE BEGIN WHILE */
121 while (1)
122 {
123     /* USER CODE END WHILE */
```



Bei der Schritt-für-Schritt Ausführung kann auch beobachtet werden, wie sich die Inhalte der Register und des Speichers ändern. In dem obigen Beispielprogramm wird zunächst der Wert 0x20000000 in das Register R0 geschrieben.



Dann wird der Wert 0x11111111 in das Register R1 geschrieben.



Danach wird der Wert 0x22222222 in das Register R2 geschrieben.

```

1.syntax unified
2.global beispiel
3.text
4.beispiel: ldr r0, =0x20000000
5           ldr r1, =0x11111111
6           ldr r2, =0x22222222
7           mov r1, r2
8           str r2, [r0]
9
10          @ Ende des Programms
11          @ Stop durch Endlos-Schleife
12.ende:    nop
13          b     ende
14

```

Name	Value
General Registers	
r0	0x20000000 (Hex)
r1	0x11111111 (Hex)
r2	0x22222222 (Hex)
r3	0x0 (Hex)
r4	0x0 (Hex)
r5	0x0 (Hex)
r6	0x0 (Hex)
r7	0x20017ff8 (Hex)
r8	0x0 (Hex)

Dann wird der Wert 0x22222222 aus dem Register R2 auch in das Register R1 kopiert.

```

1.syntax unified
2.global beispiel
3.text
4.beispiel: ldr r0, =0x20000000
5           ldr r1, =0x11111111
6           ldr r2, =0x22222222
7           mov r1, r2
8           str r2, [r0]
9
10          @ Ende des Programms
11          @ Stop durch Endlos-Schleife
12.ende:    nop
13          b     ende
14

```

Name	Value
General Registers	
r0	0x20000000 (Hex)
r1	0x22222222 (Hex)
r2	0x22222222 (Hex)
r3	0x0 (Hex)
r4	0x0 (Hex)
r5	0x0 (Hex)
r6	0x0 (Hex)
r7	0x20017ff8 (Hex)
r8	0x0 (Hex)

Schließlich wird der Wert auch an die Speicheradresse 0x20000000, die im Register R0 steht, gespeichert.

```

1.syntax unified
2.global beispiel
3.text
4.beispiel: ldr r0, =0x20000000
5           ldr r1, =0x11111111
6           ldr r2, =0x22222222
7           mov r1, r2
8           str r2, [r0]
9
10          @ Ende des Programms
11          @ Stop durch Endlos-Schleife
12.ende:    nop
13          b     ende
14

```

Name	Value
General Registers	
r0	0x20000000 (Hex)
r1	0x22222222 (Hex)
r2	0x22222222 (Hex)
r3	0x0 (Hex)
r4	0x0 (Hex)
r5	0x0 (Hex)
r6	0x0 (Hex)
r7	0x20017ff8 (Hex)
r8	0x0 (Hex)

Monitors				
0x20000000 : 0x20000000 <Hex>				
New Renderings...				
0x20000000	Address	0 - 3	4 - 7	8 - B
	20000000	22222222	00000000	01000000
	20000010	00000000	00000000	00000000
	20000020	00000000	00000000	00440040
	20000030	00000000	00000000	0C000000

C-Programmierung

Für die C-Programmierung des Nucleo-Boards wird die `main.c`-Funktion verwendet. Dabei ist zu beachten, dass eigener Code nur zwischen die `BEGIN`- und `END`-Tags geschrieben werden sollte, da Code außerhalb dieser Tags bei erneuter automatischer Code-Erzeugung (z.B. nach Änderungen in der `ioc`-Datei) gelöscht wird.

Funktion: `printf`

Um die Funktion `printf` wie gewohnt nutzen zu können, müssen ein paar Änderungen vorgenommen werden, wie unter <https://shawnhymel.com/1873/how-to-use-printf-on-stm32/> beschrieben.

Im Ordner *Inc* muss eine Datei `retarget.h` hinzugefügt werden.

`retarget.h`

```
// All credit to Carmine Noviello for this code.
// https://github.com/cnoviello/mastering-stm32/blob/
// master/nucleo-f030R8/system/include/retarget/retarget.h

#ifndef _RETARGET_H_
#define _RETARGET_H_

#include "stm32l4xx_hal.h"
#include <sys/stat.h>

void RetargetInit(UART_HandleTypeDef *huart);
int _isatty(int fd);
int _write(int fd, char* ptr, int len);
int _close(int fd);
int _lseek(int fd, int ptr, int dir);
int _read(int fd, char* ptr, int len);
int _fstat(int fd, struct stat* st);

#endif // #ifndef _RETARGET_H_
```

Im Ordner *Src* muss eine Datei *retarget.c* hinzugefügt werden.

retarget.c

```
// All credit to Carmine Novello for this code
// https://github.com/cnovello/mastering-stm32/blob/
// master/nucleo-f030R8/system/src/retarget/retarget.c

#include <_ansi.h>
#include <_syslist.h>
#include <errno.h>
#include <sys/time.h>
#include <sys/times.h>
#include <limits.h>
#include <signal.h>
#include <../Inc/retarget.h>
#include <stdint.h>
#include <stdio.h>

#if !defined(OS_USE_SEMIHOSTING)

#define STDIN_FILENO 0
#define STDOUT_FILENO 1
#define STDERR_FILENO 2

UART_HandleTypeDef *gHuart;

void RetargetInit(UART_HandleTypeDef *huart) {
    gHuart = huart;

    /* Disable I/O buffering for STDOUT stream, so that
     * chars are sent out as soon as they are printed. */
    setvbuf(stdout, NULL, _IONBF, 0);
}

int _isatty(int fd) {
    if (fd >= STDIN_FILENO && fd <= STDERR_FILENO)
        return 1;

    errno = EBADF;
    return 0;
}
```

```
int _write(int fd, char* ptr, int len) {
    HAL_StatusTypeDef hstatus;

    if (fd == STDOUT_FILENO || fd == STDERR_FILENO) {
        hstatus = HAL_UART_Transmit(gHuart, (uint8_t *) ptr, len, HAL_MAX_DELAY);
        if (hstatus == HAL_OK)
            return len;
        else
            return EIO;
    }
    errno = EBADF;
    return -1;
}

int _close(int fd) {
    if (fd >= STDIN_FILENO && fd <= STDERR_FILENO)
        return 0;

    errno = EBADF;
    return -1;
}

int _lseek(int fd, int ptr, int dir) {
    (void) fd;
    (void) ptr;
    (void) dir;

    errno = EBADF;
    return -1;
}

int _read(int fd, char* ptr, int len) {
    HAL_StatusTypeDef hstatus;

    if (fd == STDIN_FILENO) {
        hstatus = HAL_UART_Receive(gHuart, (uint8_t *) ptr, 1, HAL_MAX_DELAY);
        if (hstatus == HAL_OK)
            return 1;
        else
            return EIO;
    }
    errno = EBADF;
    return -1;
}
```

```
int _fstat(int fd, struct stat* st) {
    if (fd >= STDIN_FILENO && fd <= STDERR_FILENO) {
        st->st_mode = S_IFCHR;
        return 0;
    }

    errno = EBADF;
    return 0;
}

#endif // #if !defined(OS_USE_SEMIHOSTING)
```

Die Datei *syscalls.c* muss über einen Rechtsklick auf den Datei-Namen unter *Properties* → *C/C++ Build* → *Settings* → *Exclude Resource from Build* deaktiviert werden. Sie wird dann ausgegraut angezeigt.

In der *main.c*-Datei muss unter *Private Includes* die *retarget.h*-Datei eingebunden werden. Außerdem muss die Initialisierungsfunktion *RetargetInit()* mit passendem Parameter aufgerufen werden.

```
/* Private includes -----*/
/* USER CODE BEGIN Includes */
#include <stdio.h>
#include "retarget.h"
/* USER CODE END Includes */
```

Die Funktionen `printf` und `scanf` können dann wie gewohnt verwendet werden.